**Практика №3**

**ELTEX**

**Студент: Антипов Игорь**

**Подгруппа: №2**

**Вариант: 4**

## Раздел 1

### 1. Войдите под пользователем user1 из практики 2 (su - user1)

```
su - user1
```

### 2. Подсчитайте количество процессов, имеющих несколько потоков выполнения

```
user1@eltex-practice2-pg2-v4:~$ ps -eo pid,nlwp,comm | awk '$2 > 1' | wc -l
11
user1@eltex-practice2-pg2-v4:~$
```

Рисунок 1 – Подсчёт количества процессов с несколькими потоками.

Команда выводит количество потоков (nlwp) для всех процессов, отфильтровывает те, где потоков больше 1, и считает их количество. Всего процессов с несколькими потоками: 11.

**3. Запустите top и настройте вывод полей с информацией о процессе следующим образом:**

**• отсортируйте список процессов по убыванию значения поля VIRT, зафиксируйте результат скриншотом, повторите для полей RES и SHR, в отчете поясните, что описывают данные показатели;**

**• удалите поля VIRT, RES, SHR и добавьте поле RUSER, сделайте так, чтобы это поле было показано после поля USER, поясните отличие RUSER/USER;**

Запуск через top

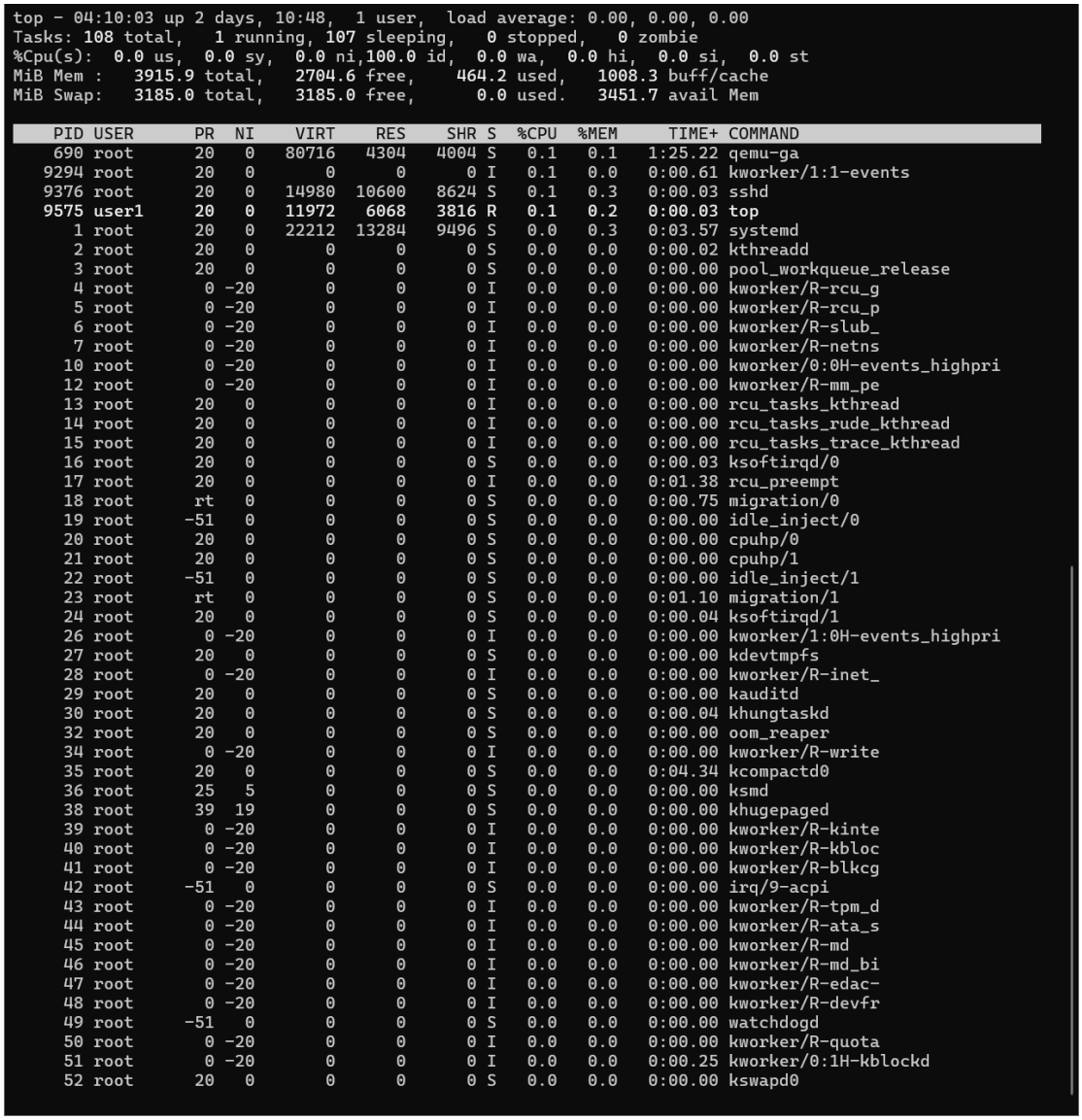



Рисунок 2 – Запуск top.

Сортировка через клавишу f

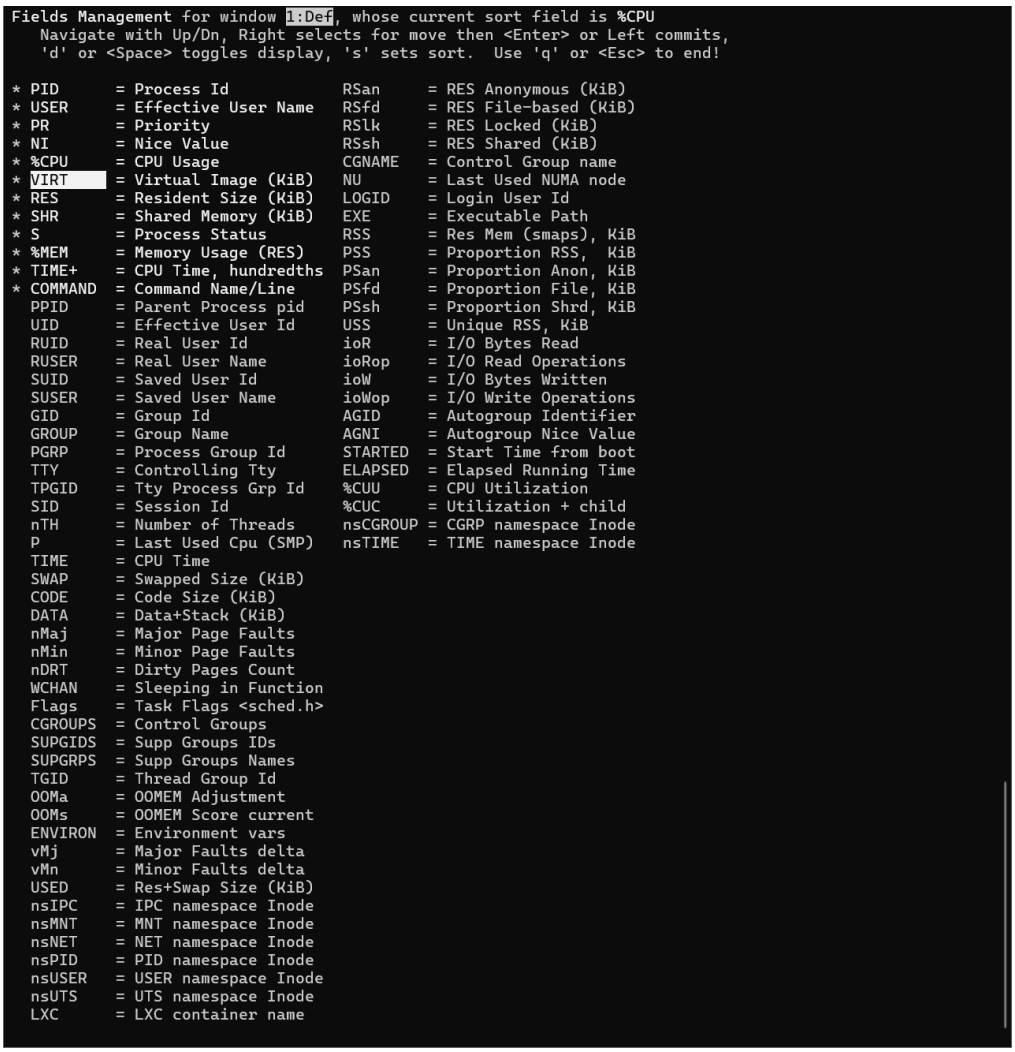



Рисунок 3 – Сортировка по VIRT (через клавишу s).

Сортировка:

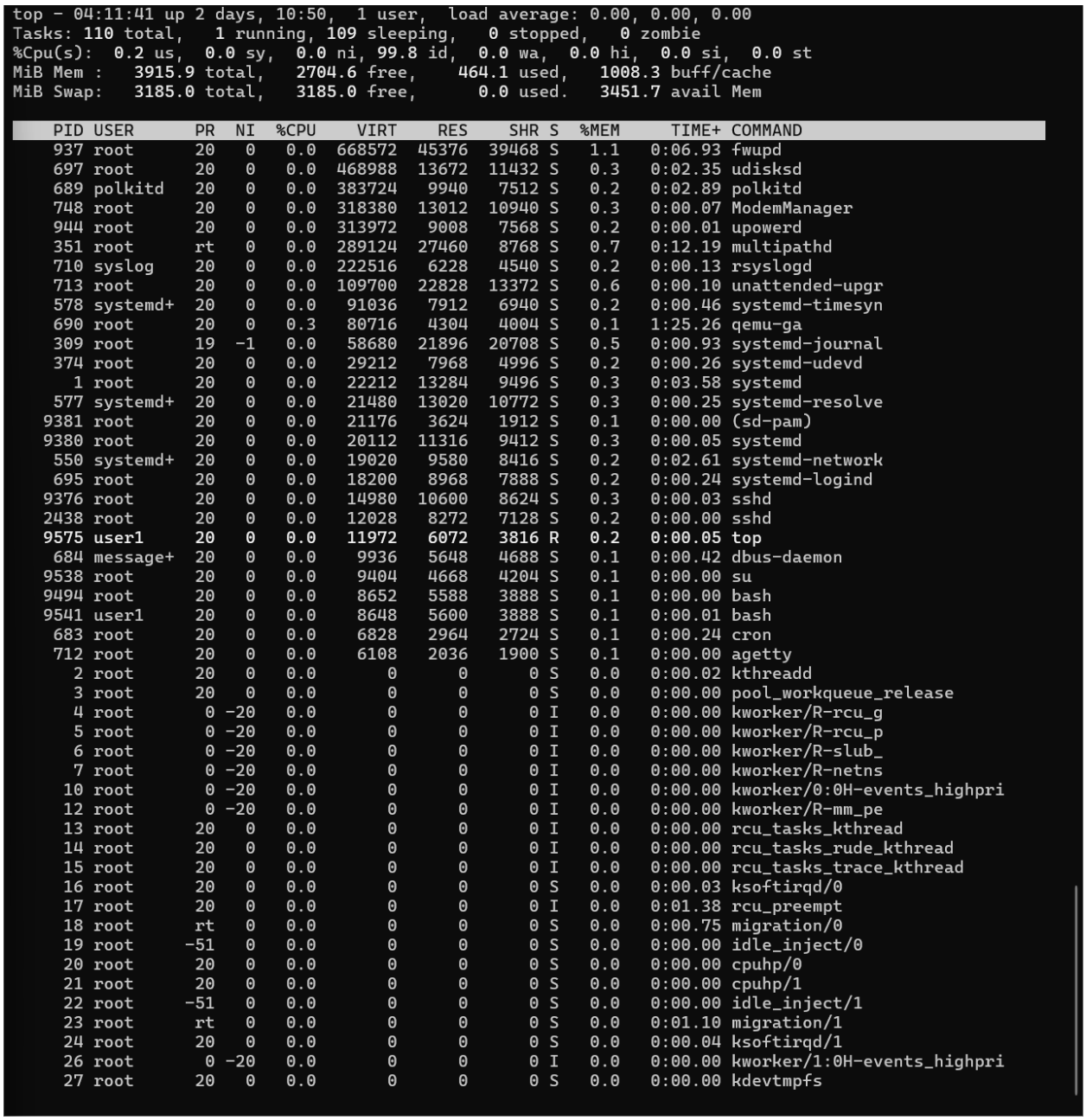



Рисунок 4 – Сортировка по VIRT.

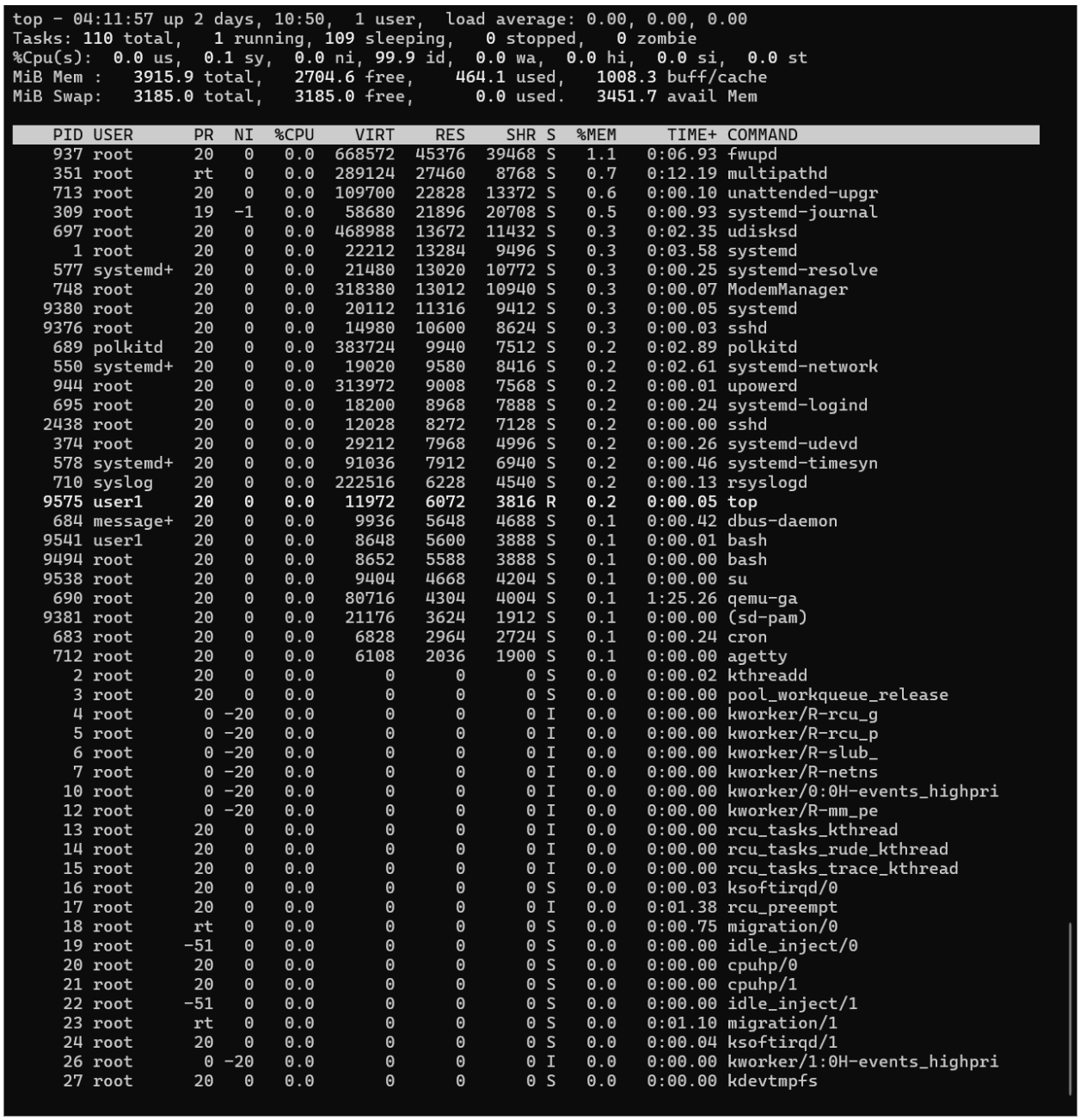



Рисунок 5 – Сортировка по RES.

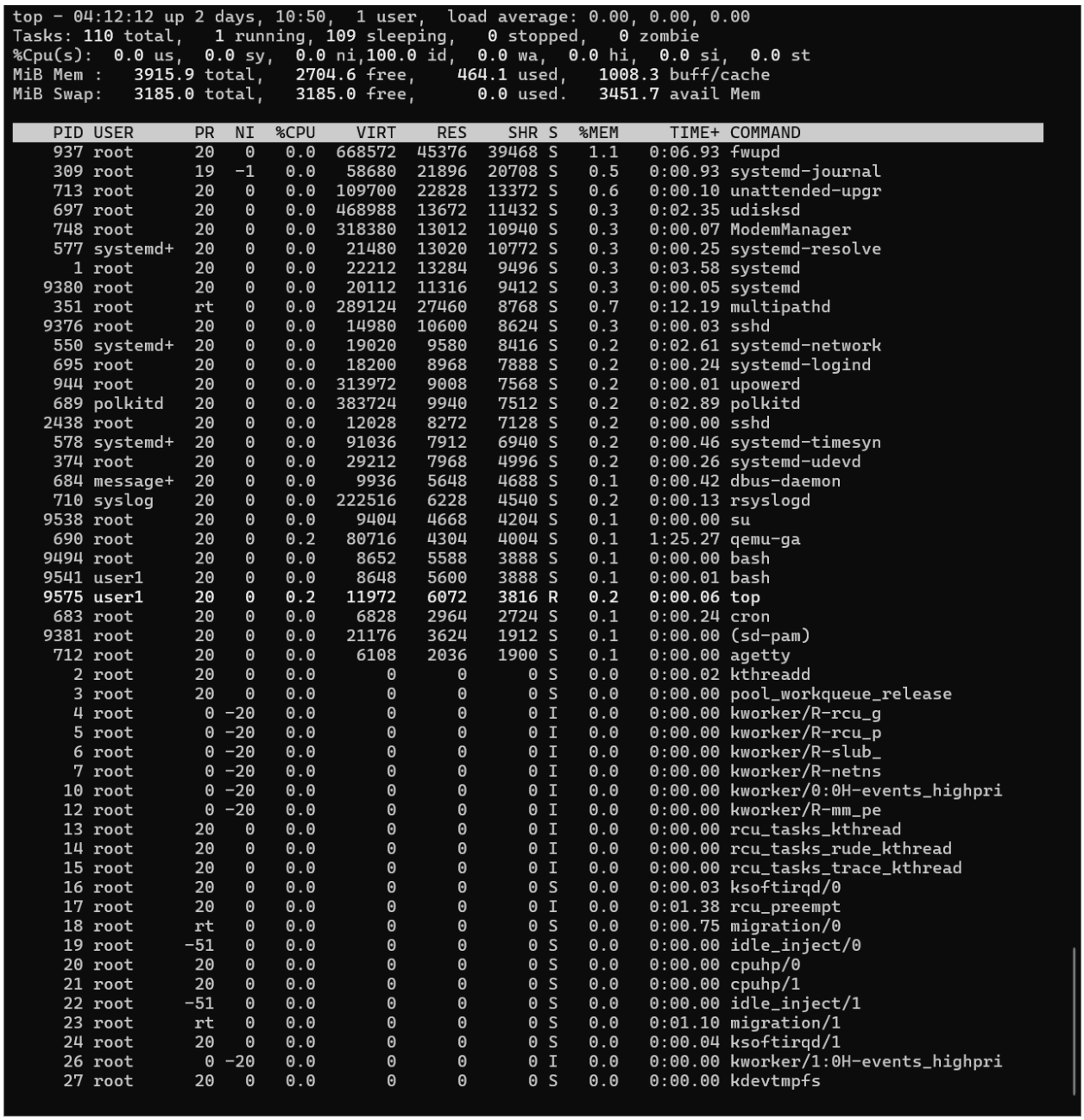



Рисунок 6 – Сортировка по RES.

VIRT (Virtual Image) — общий объем виртуальной памяти, который запросил процесс (включая код, данные и выделенную, но не используемую память). RES (Resident Size) — фактический объем физической оперативной памяти (RAM), занимаемый процессом в данный момент. SHR (Shared Memory) — объем памяти, который используется совместно с другими процессами (например, общие библиотеки).

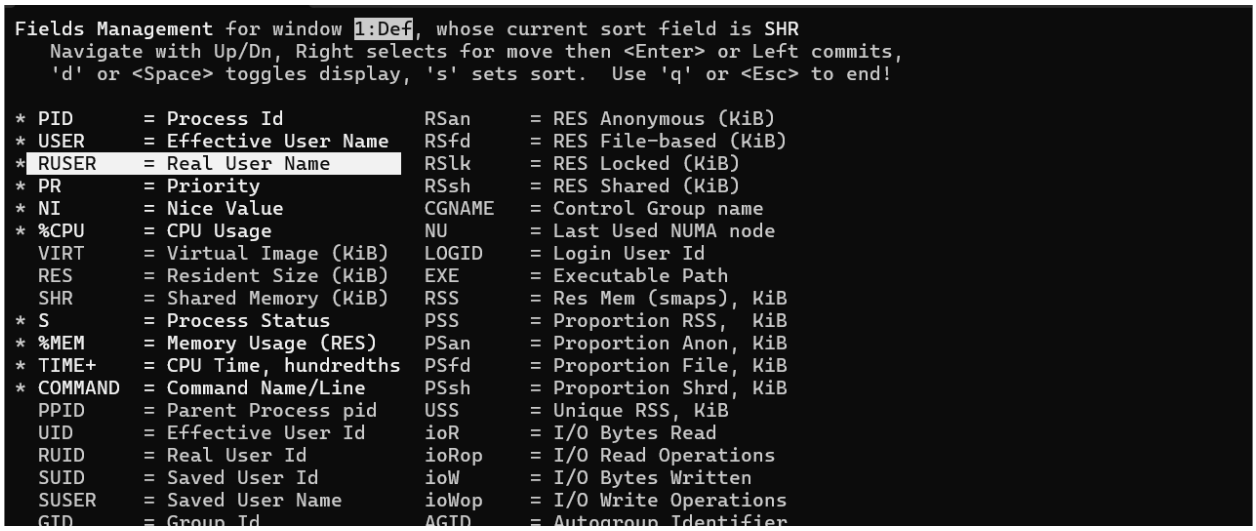



Рисунок 7 – Перемещение RUSER.

USER (Эффективный пользователь) — пользователь, от имени которого выполняется процесс. Определяет права доступа. RUSER (Реальный пользователь) — тот, кто фактически запустил процесс. Разница видна на SUID-программах (например, passwd): RUSER будет user1, а USER — root.

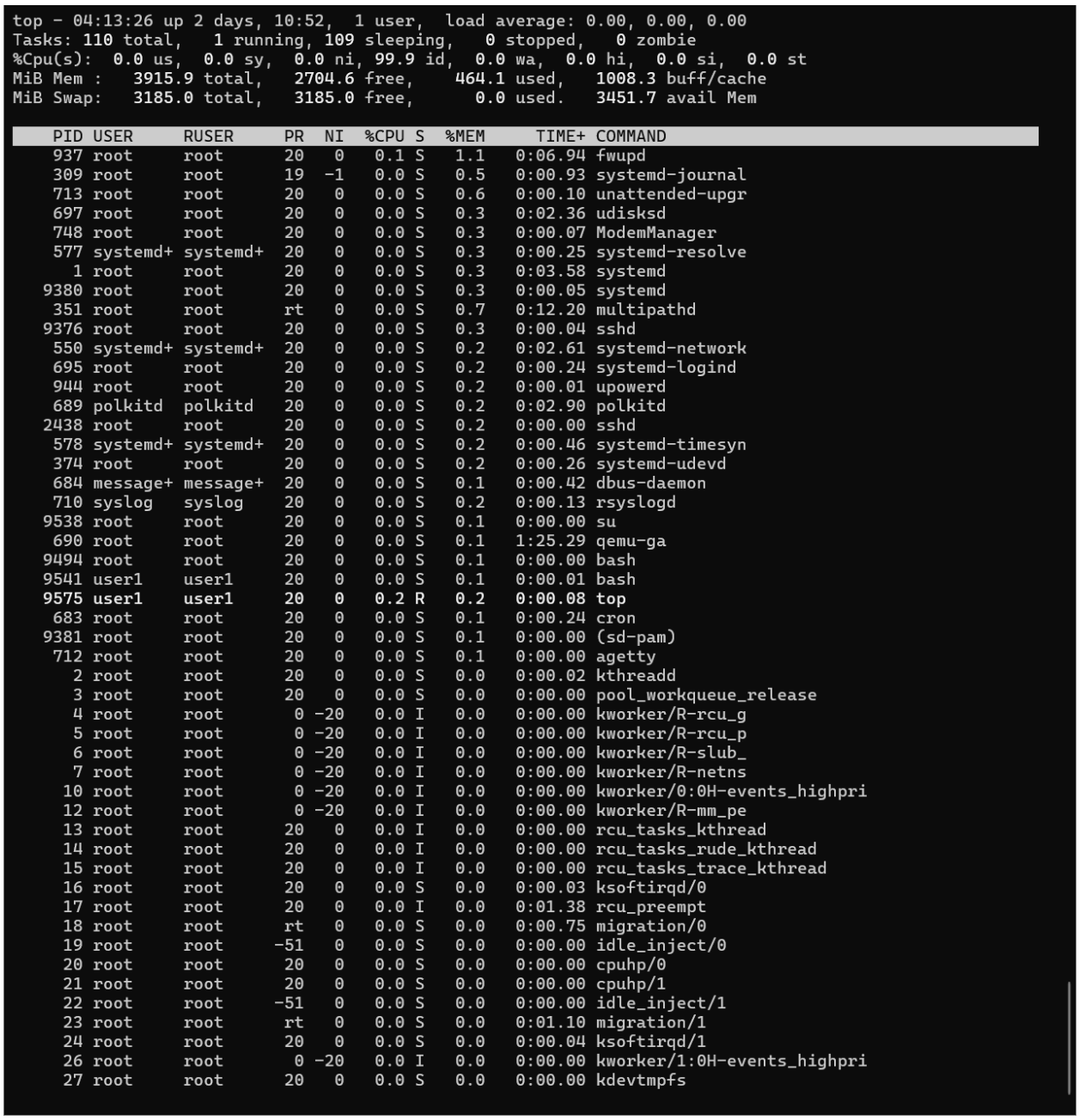



Рисунок 8 – Удалены VIRT, RES, SHR, добавлен RUSER.

**4. В другом терминальном окне выполните команду passwd и оставьте ее в состоянии запроса текущего пароля**

Выполнено

**5. Перейдите в терминальное окно с top и выполните следующие действия:**

**• выведите все процессы, для которых реальным пользователем является пользователь, которым вы вошли в сеанс;**

**• найдите процесс, запущенный командой passwd;**

**• отправьте этому процессу сигналы SIGTERM, SIGINT, SIGQUIT, SIGKILL**

**используя их числовые идентификаторы (man 7 signal)**

Для сортировки реального пользователя нажата клавиша `U` для RUSER, введено user1.

```
top - 05:21:19 up 2 days, 12:00,  2 users,  load average: 0.00, 0.00, 0.00
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.2 sy,  0.0 ni, 99.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   2695.6 free,    472.0 used,   1009.5 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3443.9 avail Mem

    PID USER      RUSER     PR  NI  %CPU S  %MEM     TIME+ COMMAND
  10482 user1     user1     20   0   0.0 S   0.1   0:00.01 bash
   9541 user1     user1     20   0   0.0 S   0.1   0:00.01 bash
   9575 user1     user1     20   0   0.0 R   0.2   0:02.85 top
  10492 root      user1     20   0   0.0 S   0.1   0:00.00 passwd
```

Рисунок 9 – Фильтрация по пользоателям.

```
top - 05:21:50 up 2 days, 12:00,  2 users,  load average: 0.00, 0.00, 0.00
Tasks: 116 total,   1 running, 115 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   3915.9 total,   2695.6 free,    472.0 used,   1009.5 buff/cache
MiB Swap:   3185.0 total,   3185.0 free,      0.0 used.   3443.9 avail Mem
Send pid 10492 signal [15/sigterm] 15
    PID USER      RUSER     PR  NI  %CPU S  %MEM     TIME+ COMMAND
  10482 user1     user1     20   0   0.0 S   0.1   0:00.01 bash
   9541 user1     user1     20   0   0.0 S   0.1   0:00.01 bash
   9575 user1     user1     20   0   0.0 R   0.2   0:02.86 top
  10492 root      user1     20   0   0.0 S   0.1   0:00.00 passwd
```

Рисунок 10 – Отправка сигнала 15.

При отправке сигналов SIGTERM (15), SIGINT (2) и SIGQUIT (3) процесс passwd не завершался. Это связано с тем, что утилита passwd защищает процесс изменения пароля от прерываний, чтобы не повредить файл /etc/shadow. Сигнал SIGKILL (9) обрабатывается на уровне ядра, его невозможно перехватить или проигнорировать, поэтому он принудительно завершил процесс.

**6. Выполните команду vim ~/file_task3.txt и нажмите Ctrl-Z**

user1@eltex-practice2-pg2-v4:~$ vim ~/file_task3.txt

Ctrl+Z

[1]+  Stopped                 vim ~/file_task3.txt

```
user1@eltex-practice2-pg2-v4:~$ vim ~/file_task3.txt

[1]+  Stopped                 vim ~/file_task3.txt
user1@eltex-practice2-pg2-v4:~$ sleep 600
^Z
[2]+  Stopped                 sleep 600
user1@eltex-practice2-pg2-v4:~$ jobs
[1]-  Stopped                 vim ~/file_task3.txt
[2]+  Stopped                 sleep 600
user1@eltex-practice2-pg2-v4:~$ bg %sleep
[2]+ sleep 600 &
user1@eltex-practice2-pg2-v4:~$ jobs
[1]+  Stopped                 vim ~/file_task3.txt
[2]-  Running                 sleep 600 &
user1@eltex-practice2-pg2-v4:~$ pgrep sleep
10500
user1@eltex-practice2-pg2-v4:~$ renice 10 -p 10500
10500 (process ID) old priority 0, new priority 10
```

Рисунок 11 – Пункты 6-9.

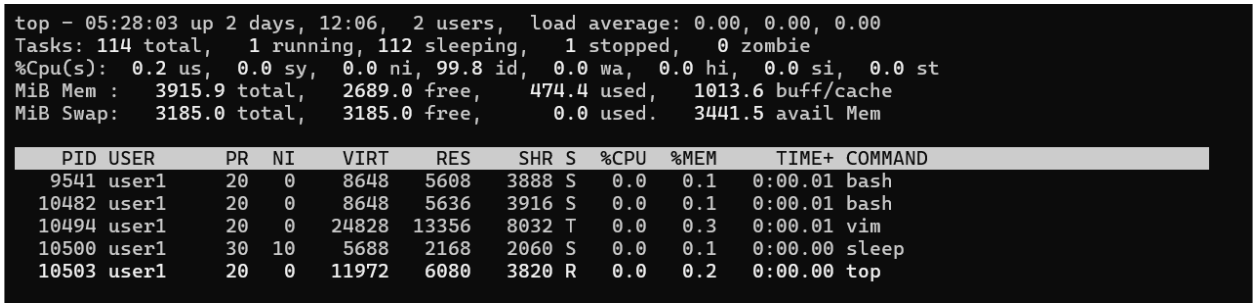



Рисунок 12 – Проверка NICE числа Пункт 10.

**11. Сделайте задание vim ~/file_ task3.txt активным и выйдите из редактора**

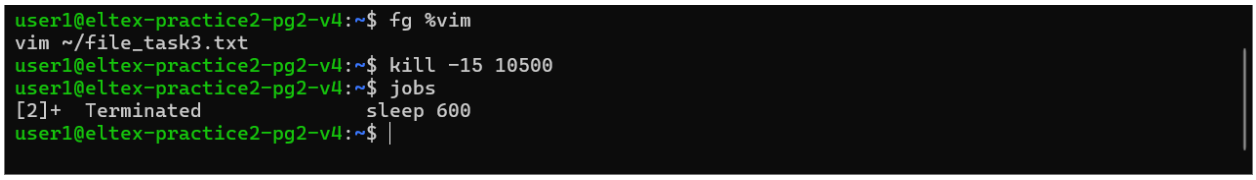



Рисунок 13 – Пункты 11-12.

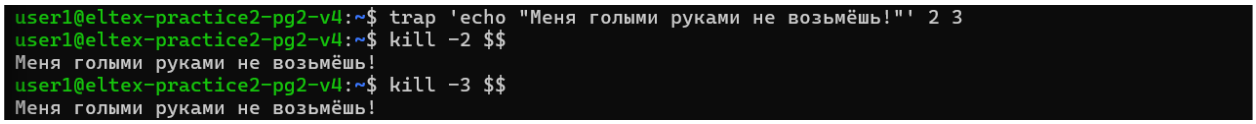



Рисунок 14 – Пункт 13.

**14. Создайте несколько зомби-процессов несколько раз выполнив команду:**

**(sleep 1 & exec /bin/sleep 30) &**

**30 – настраиваемая задержка времени существования зомби-процесса в секундах, если не успеваете выполнить следующий пункт – увеличьте время**

**15. Найдите при помощи команды ps все зомби-процессы, выведите их PID и PPID. Попробуйте досрочно завершить выполнение зомби-процессов. Все, что узнали о зомби-процессах и их особенностях в процессе выполнения пунктов опишите используя ~10 предложений в отчете.**

```
user1@eltex-practice2-pg2-v4:~$ fg %vim
vim ~/file_task3.txt
user1@eltex-practice2-pg2-v4:~$ kill -15 10500
user1@eltex-practice2-pg2-v4:~$ jobs
[2]+  Terminated              sleep 600
user1@eltex-practice2-pg2-v4:~$ trap 'echo "Меня голыми руками не возьмёшь!"' 2 3
user1@eltex-practice2-pg2-v4:~$ kill -2 $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-pg2-v4:~$ kill -3 $$
Меня голыми руками не возьмёшь!
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 60) &
[1] 10511
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 30) &
[2] 10513
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 30) &
[3] 10515
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 30) &
[4] 10517
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 30) &
[5] 10519
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 30) &
[6] 10521
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 30) &
[7] 10523
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 60) &
[8] 10525
user1@eltex-practice2-pg2-v4:~$ ps -ef | grep defunct
user1      10512   10511  0 05:31 pts/0    00:00:00 [sleep] <defunct>
user1      10526   10525  0 05:31 pts/0    00:00:00 [sleep] <defunct>
user1      10528    9541  0 05:32 pts/0    00:00:00 grep --color=auto defunct
[2]   Done                    ( sleep 1 & exec /bin/sleep 30 )
[3]   Done                    ( sleep 1 & exec /bin/sleep 30 )
[4]   Done                    ( sleep 1 & exec /bin/sleep 30 )
[5]   Done                    ( sleep 1 & exec /bin/sleep 30 )
[6]   Done                    ( sleep 1 & exec /bin/sleep 30 )
[7]-  Done                    ( sleep 1 & exec /bin/sleep 30 )
user1@eltex-practice2-pg2-v4:~$
```

Рисунок 15 – Выполнение пункто 14-15.

```
user1@eltex-practice2-pg2-v4:~$ (sleep 1 & exec /bin/sleep 600) &
[1] 10538
user1@eltex-practice2-pg2-v4:~$ ps -eo pid,ppid,stat,cmd | grep Z
  10539   10538 Z    [sleep] <defunct>
  10541    9541 S+   grep --color=auto Z
user1@eltex-practice2-pg2-v4:~$ kill -9 10539
user1@eltex-practice2-pg2-v4:~$ ps -eo pid,ppid,stat,cmd | grep Z
  10539   10538 Z    [sleep] <defunct>
  10543    9541 S+   grep --color=auto Z
user1@eltex-practice2-pg2-v4:~$ kill -9 10543
-bash: kill: (10543) - No such process
user1@eltex-practice2-pg2-v4:~$ ps -eo pid,ppid,stat,cmd | grep Z
  10539   10538 Z    [sleep] <defunct>
  10545    9541 S+   grep --color=auto Z
user1@eltex-practice2-pg2-v4:~$ kill -9 10538
user1@eltex-practice2-pg2-v4:~$ ps -eo pid,ppid,stat,cmd | grep Z
  10547    9541 S+   grep --color=auto Z
[1]+  Killed                  ( sleep 1 & exec /bin/sleep 600 )
user1@eltex-practice2-pg2-v4:~$
```

Рисунок 16 – Повторное «убийство» зомби-процессов.

Зомби-процесс — это процесс, который уже завершил свою работу, но его запись всё ещё хранится в таблице процессов. Это происходит потому, что родительский процесс еще не прочитал код возврата своего дочернего процесса с помощью системного вызова wait(). Зомби не потребляют ресурсы процессора или оперативной памяти, но занимают слот в таблице процессов. Найти зомби можно с помощью команды ps, они помечаются статусом Z (или defunct). Особенность зомби в том, что их невозможно убить стандартным сигналом SIGKILL (9), так как они уже мертвы. В ходе эксперимента я

попытался убить зомби командой kill -9, но система выдала ошибку "No such process". Чтобы избавиться от зомби, необходимо завершить его родительский процесс. Когда родитель умирает, зомби переназначается процессу init (PID 1), который автоматически собирает его статус и освобождает ресурсы. Опасность зомби в том, что если родительская программа работает долго и создает их слишком много, таблица процессов может переполниться. Поэтому в норме родительские процессы всегда должны корректно обрабатывать завершение своих дочерних процессов.

## Раздел 2

**1. Создайте скрипт на языке bash с именем cuckoo.sh, делающий следующее:**

- **Создает именованный канал с именем, указанном с абсолютным путем в параметре запуска скрипта --pishitesuda=**
- **Записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Startup!**
- **В процессе ожидания читает сообщения из именованного канала, на запрос «NAME[PID]: how much time do I have?" возвращает число N в диапазоне от 2 до 10, где NAME – имя обратившегося процесса и PID – его идентификатор, записывает обращение и ответ в файл cuckoo.log в формате: ДАТА ВРЕМЯ NAME[PID] N**
- **При получении сигнала (на ваш выбор) скрипт записывает в файл cuckoo.log сообщение в формате: ДАТА ВРЕМЯ Shutdown! и завершает свою работу, предварительно удалив именованный канал.**

Файл cuckoo.sh:

```
#!/bin/bash

LOG_FILE="cuckoo.log"

PIPE_PATH=""

for arg in "$@"; do

  if [[ "$arg" == --pishitesuda=* ]]; then PIPE_PATH="${arg#*=}"; fi

done

[ ! -p "$PIPE_PATH" ] && mkfifo "$PIPE_PATH"


echo "$(date '+%Y-%m-%d %H:%M:%S') Startup!" >> "$LOG_FILE"

trap 'echo "$(date "+%Y-%m-%d %H:%M:%S") Shutdown!" >> "$LOG_FILE"; rm -f "$PIPE_PATH"; exit 0' SIGTER>


while true; do

    if read -r line < "$PIPE_PATH"; then

        if [[ "$line" == *"how much time do I have?"* ]]; then

            NAME_PID="${line%%:*}"

            N=$(( RANDOM % 9 + 2 ))

            echo "$(date '+%Y-%m-%d %H:%M:%S') $NAME_PID $N" >> "$LOG_FILE"
```

```
            echo "$N" > "$PIPE_PATH"
        fi
    fi
done
```

Выдача права на выполнение:

```
chmod +x cuckoo.sh
```

**2. Создайте службу systemd (unit) для запуска скрипта cuckoo.sh, разрешите его работу (enable), запустите его и проверьте состояние (status)**

```
sudo nano /etc/systemd/system/cuckoo.service
```

```
[Unit]
Description=Cuckoo Script Service
After=network.target

[Service]
Type=simple
WorkingDirectory=/home/user1/r2
ExecStart=/home/user1/r2/cuckoo.sh --pishitesuda=/tmp/cuckoo_pipe
Restart=on-failure
User=user1
Group=user1

[Install]
WantedBy=multi-user.target
```

```
user1@eltex-practice2-pg2-v4:~/r2$ sudo systemctl daemon-reload
user1@eltex-practice2-pg2-v4:~/r2$ sudo systemctl start cuckoo.service
user1@eltex-practice2-pg2-v4:~/r2$ sudo systemctl status cuckoo.service
● cuckoo.service - Cuckoo Script Service
     Loaded: loaded (/etc/systemd/system/cuckoo.service; disabled; preset: enabled)
     Active: active (running) since Wed 2026-10-21 07:07:31 UTC; 3s ago
   Main PID: 18728 (cuckoo.sh)
      Tasks: 1 (limit: 4600)
     Memory: 352.0K (peak: 1.0M)
        CPU: 1ms
     CGroup: /system.slice/cuckoo.service
             └─18728 /bin/bash /home/user1/r2/cuckoo.sh --pishitesuda=/tmp/cuckoo_pipe

Oct 21 07:07:31 eltex-practice2-pg2-v4 systemd[1]: Started cuckoo.service - Cuckoo Script Service.
```

Рисунок 17 – Запуск сервиса.

Для применения конфигурации и запуска службы были выполнены команды: sudo systemctl daemon-reload, sudo systemctl enable cuckoo.service, sudo

systemctl start cuckoo.service. Проверка статуса подтвердила, что служба активна: active (running).

3. **Создайте скрипт на языке bash с именем template_task.sh, делающий следующее:**
   - **При запуске проверят, что имя скрипта не совпадает с template_task.sh, если совпадает - выходит с уведомлением «я бригадир, сам не работаю»**
   - **При запуске дописывает в файл report_имя_скрипта_без_полного_пути.log в рабочем каталоге информацию: ДАТА ВРЕМЯ [PID] Скрипт запущен**
   - **Запрашивает через именованный канал скрипта cuckoo.sh, который получает из procfs, время сообщением в указанном ранее формате и ждет указанное количество секунд.**
   - **Дописывает в файл report_имя_скрипта_без_полного_пути.log сообщение: ДАТА ВРЕМЯ [PID] Скрипт завершился, работал N секунд.**

Файл template_task.sh:

```
#!/bin/bash

SCRIPT_NAME=$(basename "$0")

if [ "$SCRIPT_NAME" = "template_task.sh" ]; then echo "я бригадир, сам не работаю"; exit 1; fi


LOG_FILE="report_${SCRIPT_NAME}.log"

PIPE_PATH="/tmp/cuckoo_pipe"


echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт запущен" >> "$LOG_FILE"


echo "${SCRIPT_NAME}[$$]: how much time do I have?" > "$PIPE_PATH" &

read -r N < "$PIPE_PATH"

[ -z "$N" ] && N=2


sleep "$N"

echo "$(date '+%Y-%m-%d %H:%M:%S') [$$] Скрипт завершился, работал $N секунд." >> "$LOG_FILE"
```

Выдача прав на выполнение

```
chmod +x template_task.sh
```

**4. Создайте скрипт на языке bash с именем observer.sh, читающий файл конфигурации со списком скриптов observer.conf, проверяющим их наличие в списке работающих процессов поиском в /proc и запускающих их в отключенном от терминала режиме (nohup) в случае отсутствия в нем. Информация о перезапуске дописывайте в файл observer.log**

Файл: observer.sh

```
#!/bin/bash

while read -r task_name; do

    [ -z "$task_name" ] && continue

    if ! pgrep -f "$task_name" > /dev/null; then

        [ ! -L "$task_name" ] && ln -s template_task.sh "$task_name"

        nohup "./$task_name" > /dev/null 2>&1 &

        echo "$(date '+%Y-%m-%d %H:%M:%S') Перезапуск: $task_name" >> observer.log

    fi

done < observer.conf
```

Выдача прав на выполнение

```
chmod +x observer.sh
```

**5. Настройте запуск observer.sh посредством cron по расписанию – 1 раз в минуту**

Подготовка символьных ссылок:

```
ln -s template_task.sh worker1

ln -s template_task.sh worker2

echo -e "worker1\nworker2" > observer.conf
```

```
user1@eltex-practice2-pg2-v4:~/r2$ (crontab -l 2>/dev/null; echo "* * * * * /home/user1/r2/observer.sh"
) | crontab -
```

Рисунок 18 – Добавление задачи в планировщик cron.

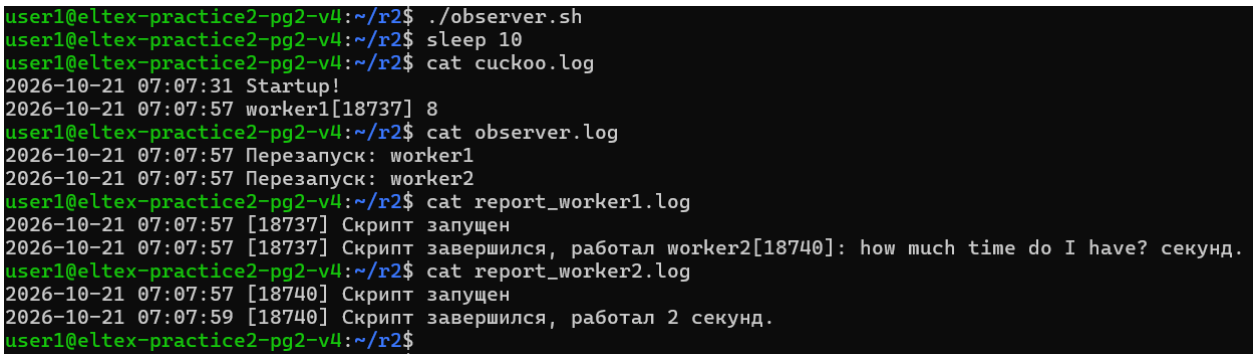



Рисунок 19 – Запуск и сборка стстистики.

В логе cuckoo.log видно, что воркеры обратились к каналу и получили время (8 и 2 секунды). В observer.log зафиксирован перезапуск задач. В логах report_worker1.log и report_worker2.log видно корректный запуск и завершение работы скриптов.

Сборка в архив и завершение процессов:

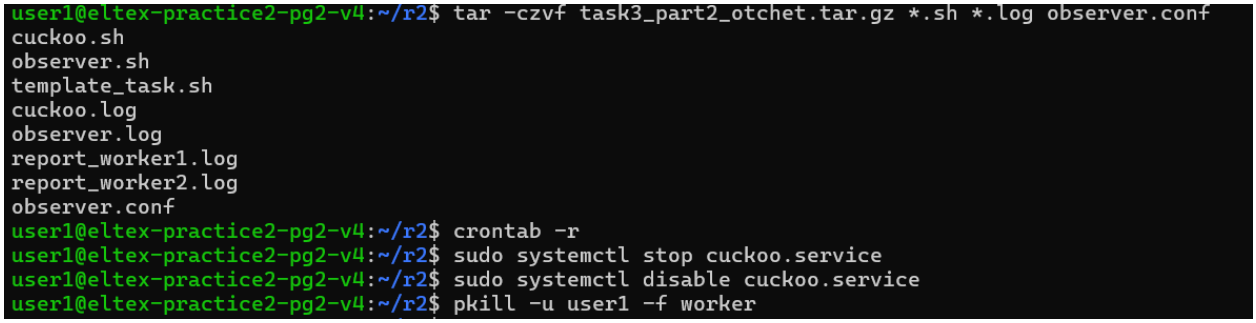



Рисунок 20 – Сборка в архив и завершение процессов.

Задача из cron была удалена командой crontab -r. Служба остановлена и отключена командами sudo systemctl stop/disable cuckoo.service. Фоновые процессы убиты.